MASTERCLASS SUNUMU

Flutter'da Yerleşim Düzenleri ve Uyarlanabilir Tasarım

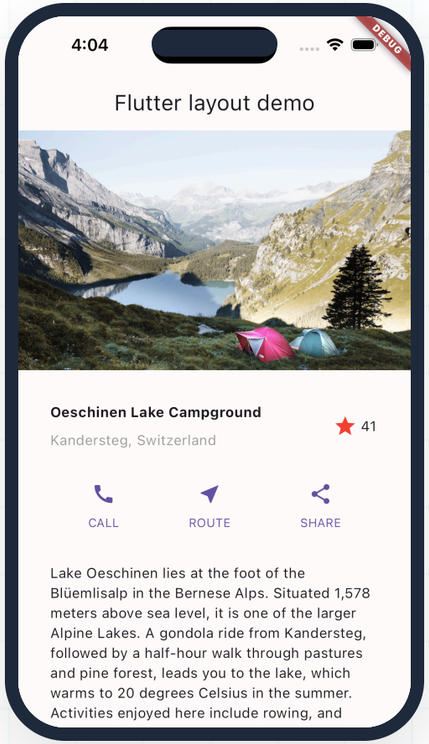
Widget Ağacından Slivers Mimarisine, Çoklu Form Faktörlerinden Platform Adaptasyonuna Kapsamlı Rehber

Modern Layout Mimarisi
Row, Column, Expanded, Stack ve Flex sistemlerinin derinlemesine incelenmesi.

Vaka Analizleri & Uygulama
Strawberry Pavlova kartı ve Lake Oeschinen arayüz inşası metodolojisi.

İleri Kaydırma & Slivers
ListView optimizasyonları, CustomScrollView ve Flow ile 60 FPS Paralaks efekti.

Uyarlanabilir & Büyük Ekranlar
Mobil, katlanabilir cihazlar, tablet ve masaüstü için responsive/adaptive prensipleri.



Flutter ile Üretilen Canlı UI Arayüzü

İçindekiler: Konu Başlıkları ve Bölüm Akışı

Resmi dokümantasyon temelinde 4 ana mimari modül (40 Slayt Kapsamı)

Bölüm 01: Temel Yerleşim Mekanizması & Eksenler Slayt 03 - 08

Widget ağacı, 3 altın kural (Constraints down, sizes up), Row/Column eksenleri, hizalama ve Expanded.

Bölüm 02: Vaka Analizleri & Adım Adım İnşa Slayt 09 - 15

Strawberry Pavlova kartı, Container kutu modeli, 4 adımlı inşa metodu ve Lake Oeschinen tutorial projesi.

Bölüm 03: İleri Seviye Listeler, Slivers & Paralaks Slayt 16 - 27

ListView.builder tembel yükleme, 2D GridView, Spaced Items, CustomScrollView ve Flow ile 60 FPS paralaks.

Bölüm 04: Uyarlanabilirlik & Platform Adaptasyonu Slayt 28 - 40

Google 3 adımlı modeli, Window Size Classes, katlanabilirler, fare/klavye girdileri, VisualDensity ve Android/iOS.

BÖLÜM 01 // GİRİŞ

01

Flutter Yerleşim Mekanizması & Widget Ağacı

'Her şey bir widget'tır' anlayışı, görünür ve görünmeyen düzenleyici öğeler

Görünür Bileşenler

Metin, buton, görsel ve simge gibi kullanıcıyla doğrudan etkileşime giren arayüz elemanları.

Görünmeyen Düzenleyiciler

Row, Column, Center, Padding ve Expanded gibi yerleşimi ve geometrik sınırları yöneten widget'lar.

Kompozisyon İlkesi

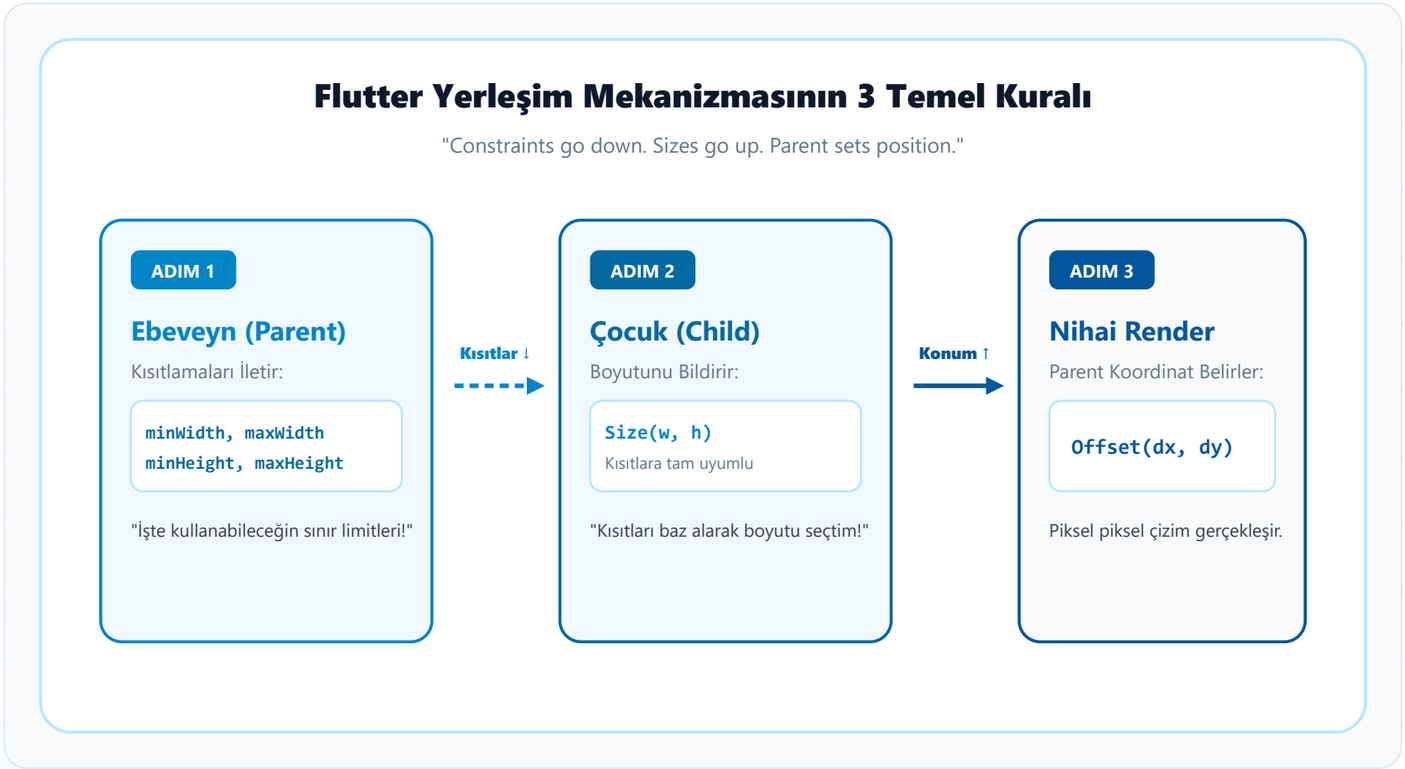
Kalıtım yerine iç içe geçme (composition) ile esnek ve modüler hiyerarşiler kurma prensibi.

Deklaratif Arayüz

Arayüz durumun (State) doğrudan bir yansımasıdır: UI = f(state).

Flutter Yerleşim Mekanizmasının 3 Altın Kuralı

'Constraints go down. Sizes go up. Parent sets position.' ilkesi



1. Kısıtlamalar Aşağı İner — Kural 1

Üst widget (Parent), alt widget'a (Child) kullanabileceği min/max genişlik ve yükseklik sınırlarını iletir.

2. Boyutlar Yukarı Çıkar — Kural 2

Alt widget kısıtlamalara uyarak kendi boyutunu (Size) belirler ve üst widget'a bildirir.

3. Konumu Üst Belirler — Kural 3

Üst widget, alt widget'ın bildirdiği boyutu alarak onu ekranda x, y koordinatlarına yerleştirir.

Tek Seferde Render O(N) — 60 FPS

Çift yönlü kısıtlama iletişimi sayesinde arayüz O(N) karmaşıklığında, tek geçişte çizilir.

Tek Bir Widget'ı Ekrana Yerleştirme

MaterialApp, Scaffold ve Center ile ilk arayüzün inşası

MaterialApp Kökü

Tema, navigasyon ve uluslararasılaştırma gibi temel altyapıyı sağlayan ana çatı bileşenidir.

Scaffold İskeleti

AppBar, Drawer, BottomNavigationBar ve FloatingActionButton için hazır yuvalar sunar.

Center Hizalaması

Üstünden gelen sınırsız kısıtlamaları alıp çocuğunu ekranın tam merkezine odaklar.

Const Constructor

Best Practice

Değişmeyen widget ağaçlarında bellek tahsisini sıfırlayan ve performansı artıran derleme sabiti.

hello_world.dart

DART

```dart
import 'package:flutter/material.dart';

void main() => runApp(const MyApp());

class MyApp extends StatelessWidget {
  const MyApp({super.key});

  @override
  Widget build(BuildContext context) {
    return MaterialApp(
      home: Scaffold(
        appBar: AppBar(title: const Text('Flutter Layout')),
        body: const Center(
          child: Text(
            'Hello World',
            style: TextStyle(fontSize: 28, color: Colors.blue),
          ),
        ),
      ),
    );
  }
```

Çoklu Widget Yerleşimi: Row ve Column Temelleri

Yatay ve dikey eksenlerde birden fazla elemanı sıralama

Row (Satır)

Altındaki widget'ları soldan sağa yatay eksende (X ekseni) yan yana sıralar.

Column (Sütun)

Altındaki widget'ları yukarıdan aşağıya dikey eksende (Y ekseni) alt alta dizer.

children: <Widget>[] Dizisi

Her iki widget da bir alt öğe dizisi kabul eder; bu öğeler de kendi içlerinde Row veya Column olabilir.

Kullanım Yaygınlığı Temel Taş

Flutter UI dünyasında en sık başvurulan iki temel yerleşim kalıbıdır.



Dikey Eksen (Column) Görsel Öğeleri

Eksenler, Hizalama ve Boyut: MainAxis & CrossAxis

Satır ve sütun geometrisi, spaceEvenly boşluk dağılımı ve MainAxisSize.min

Ana Eksen (Main Axis)

Row için yatay (X), Column için dikeydir (Y). MainAxisAlignment ile öğelerin akış yönündeki konumu belirlenir.

Boşluk Dağılımı (spaceEvenly / Between) Boşluk

spaceEvenly öğelerin arasına ve kenarlara eşit boşluk bırakır; spaceBetween ise ilk ve son öğeyi uçlara yaslar.

Çapraz Eksen (Cross Axis)

Ana eksene 90 derece dik yöndür (start, center, end, stretch hizalaması).

MainAxisSize.min Sıkıştırması Sıkıştırma

Varsayılan max yayılmayı önler; Row/Column'ın yalnızca çocuklarının kapladığı alan kadar yer tutmasını sağlar.

ÖZELLİK	ROW (SATIR)	COLUMN (SÜTUN)
Ana Akış Yönü	Yatay (Horizontal: X ekseni) Row	Dikey (Vertical: Y ekseni) Column
MainAxisAlignment	Genişlik boyunca dağıtır (start, center, spaceEvenly)	Yükseklik boyunca dağıtır (start, center, spaceEvenly)
CrossAxisAlignment	Dikeyde hizalar (üst, orta, alt, stretch)	Yatayda hizalar (sol, orta, sağ, stretch)
Eksen Boyut Modu	MainAxisSize.min ile yalnızca öğeler kadar daralır	MainAxisSize.max ile dikeyde tüm boş alanı kaplar

Boyutlandırma & Taşma Hataları: Expanded & Flex

Sarı-siyah çizgili Overflow hatası ve esnek boyutlandırma çözümü

Taşma (Overflow) Hatası

⚠ RIGHT OVERFLOWED BY 580 PIXELS



Görsellerin veya metinlerin toplam genişliği ekranı aştığında sarı-siyah şeritli hata belirir ve sağ kenar kesilir.

Expanded ile Esnek Dağılım

✓ ESNEK KOD ÇÖZÜMÜ

flex_solution.dart — DART

```dart
Row(
  crossAxisAlignment: CrossAxisAlignment.center,
  children: [
    Expanded(
      child: Image.asset('images/pic1.jpg'),
    ),
    Expanded(
      flex: 2, // 2 kat daha geniş
      child: Image.asset('images/pic2.jpg'),
    ),
    Expanded(
      child: Image.asset('images/pic3.jpg'),
    ),
  ],
);
```

Elemanları Expanded ile sarmalamak ekran sınırlarına zorlar. flex: 2 katsayısı ile ortadaki görsel iki kat alan kazanır.

BÖLÜM 02 // GİRİŞ

02

Vaka Analizleri & Adım Adım Uygulama İnşası

Strawberry Pavlova kartı, modüler mimari, Container kutu modeli ve Lake Oeschinen arayüz inşası

Strawberry Pavlova Kartı

İç içe geçmiş satır ve sütunların gerçek dünya uygulaması.

Modüler Değişken Mimarisi

Büyük widget ağaçlarını bağımsız değişkenlere ve fonksiyonlara ayırma.

Container Kutu Modeli

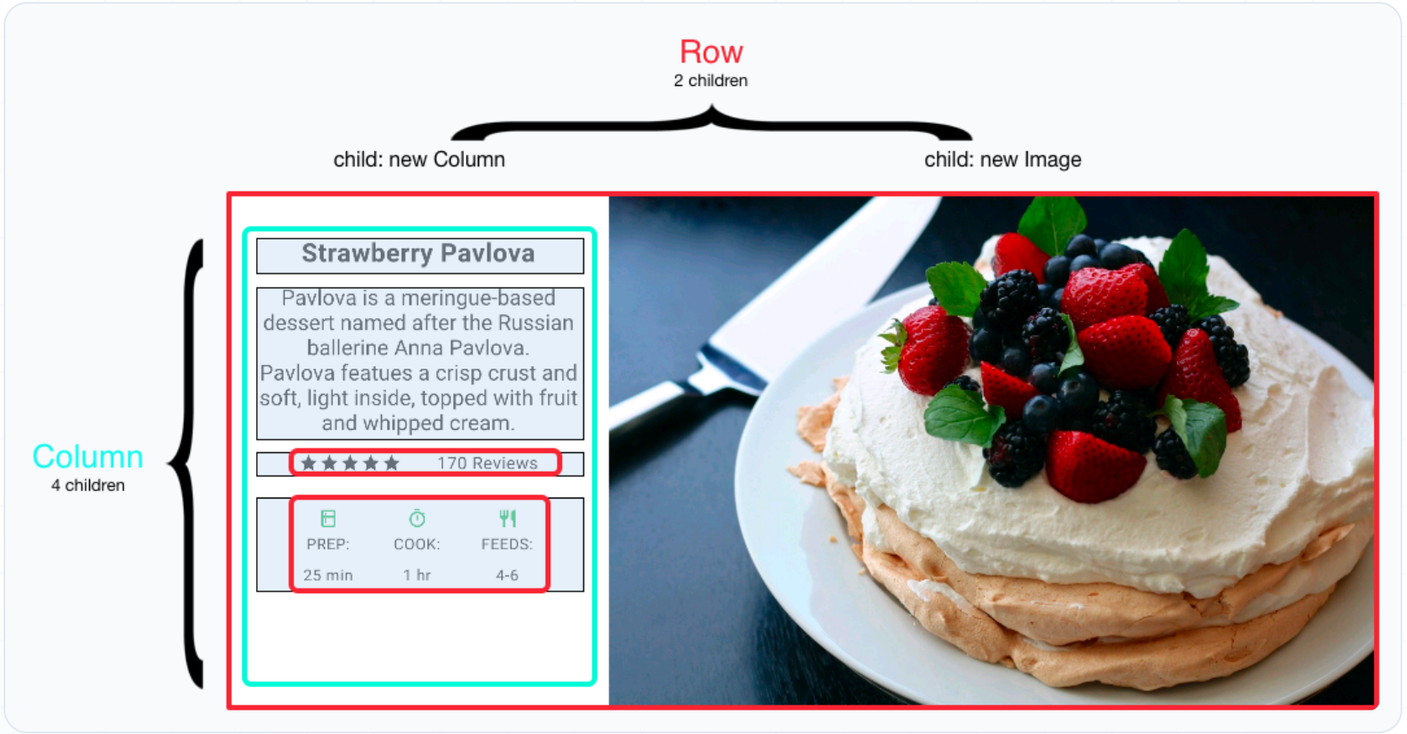
Padding, Margin, Border ve BoxDecoration ile stil katmanı.

Oeschinen Lake Projesi

Başlık, düğmeler, metin ve resim bölümlerinin adım adım inşası.

Strawberry Pavlova: Kompleks Kart Mimarisi & Şema

İç içe satır/sütunlar, ratings ve ikon bloklarının modüler çözümlemesi



Ana Yerleşim (Row) `Row(1:1)`

Solda detayları içeren bir sütun (leftColumn), sağda ise tam boy görsel (mainImage) 1:1 dengede durur.

Puanlama Bölümü (Ratings)

Yıldız ikonlarını barındıran alt Row ve '170 Reviews' metni dengeli aralıklarla yerleştirilir.

İkon ve İstatistik Satırı `Modüler`

PREP, COOK ve FEEDS başlıkları ikon + metin sütunları olarak satıra dizilir; DefaultTextStyle.merge ile ortak stil aktarılır.

Card & SizedBox Sınırı

Arayüz bir Card içine yerleştirilir ve genişliği SizedBox(width: 440) ile masaüstü/tablette sınırlandırılır.

Material Bileşenleri: Card ve ListTile

Hazır semantik yapılar ve derinlik hissi veren kart tasarımları

Card Widget'ı

İlgili bilgileri yuvarlatılmış köşeler ve gölgeyle (elevation) sarmalayarak 3B yüzey hissi verir.

elevation Değeri

3D Derinlik

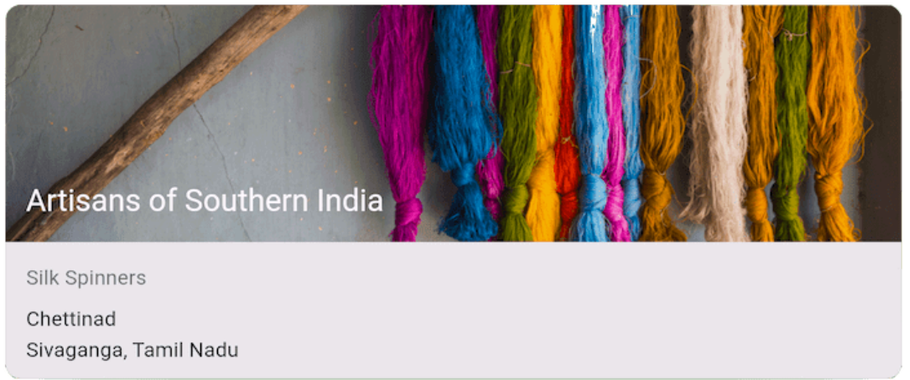
Kartın sanal ışık kaynağından ne kadar yükseleceğini ve gölgenin yayılımını kontrol eder.

ListTile Standardı

leading (baş ikon), title (başlık), subtitle (alt başlık) ve trailing alanlarına sahip hazır satırdır.

Kombinasyon Gücü

Bir Card içinde Column ve Divider ile ayrılmış ListTile'lar kurumsal listeler için idealdir.



Material Card ve ListTile Görünümü

Yerleşim Düzenini Tasarlama: 4 Adımlı İnşa Metodolojisi

'İki kez ölç, bir kez kes' yaklaşımıyla arayüzü analitik bloklara ayrıştırma

1

Temel Eksenleri Çizin

Arayüzün dikey mi yoksa yatay mı aktığını belirleyin. Row ve Column sınırlarını işaretleyin.

Adım 1: Eksenler

2

Makro Bloklara Parçalayın

Ekranı bağımsız 4 ana gövdeye bölün: Image, Title, Button, Text bölümleri.

Adım 2: Parçalama

3

Modüler Widget'lar Yazın

Her bölümü bağımsız bir StatelessWidget olarak kodlayın. Kendi iç boşluğunu kendi yönetsin.

Adım 3: İzolasyon

4

Ana Gövdede Birleştirin

Tüm parçaları tek bir Column altında toplayın ve SingleChildScrollView ile taşmayı önleyin.

Adım 4: Montaj

Başlık ve Düğme Bölümlerinin İnşası (TitleSection & ButtonSection)

Expanded ile başlık alanı esnetme ve ButtonWithText bileşeni

TitleSection Mimarisi

Sol taraftaki metin sütunu Expanded ile sarılarak satırın geri kalanını kaplar; sağda yıldız ve 41 sayısı yer alır.

ButtonWithText Tasarımı

İkon ve altındaki etiketi tek bir Column içinde toplayan tekrar kullanılabilir özel widget.

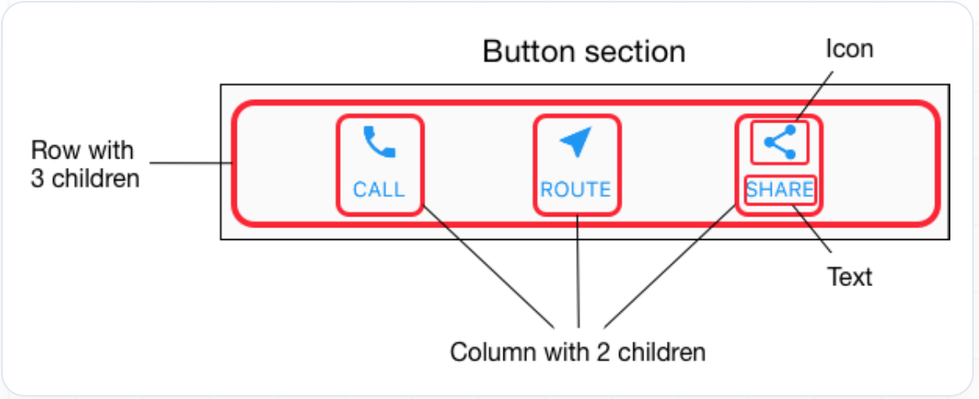
spaceEvenly Düğme Satırı

CALL, ROUTE ve SHARE butonları Row içinde eşit aralıklarla konumlandırılır.

Theme.of(context) Entegrasyonu

Tema

Düğme renkleri uygulamanın birincil renginden (primaryColor) dinamik olarak alınır.



ButtonSection Modüler Şeması

Metin ve Medya: ImageSection, Varlık Yönetimi ve BoxFit

pubspec.yaml yapılandırması, BoxFit.cover ve softWrap kuralı

pubspec.yaml Assets Bildirimi

Görseller proje kökündeki 'images/' dizinine eklenmeli ve pubspec.yaml içinde tanımlanmalıdır.

BoxFit.cover

Görseli orijinal en-boy oranını bozmadan ayırılan render kutusunun tamamını dolduracak şekilde kırpar.

TextSection softWrap

softWrap: true ile metin ekran kenarına ulaştığında kelime sınırlarından düzgünce alt satıra geçer.

SingleChildScrollView Gövdesi

Scroll

Ekran boyutu yetersiz kaldığında taşmayı önlemek için ana sütun kaydırılabilir bir görünüme sarılır.

image_section.dart — DART

```dart
// pubspec.yaml
flutter:
  uses-material-design: true
  assets:
    - images/lake.jpg

// Dart Kodu
class ImageSection extends StatelessWidget {
  const ImageSection({super.key, required this.image});
  final String image;

  @override
  Widget build(BuildContext context) {
    return Image.asset(
      image,
      width: 600,
      height: 240,
      fit: BoxFit.cover,
    );
  }
}
```

Tamamlanan Uygulama: Oeschinen Lake Campground

Modüler bileşenlerin SingleChildScrollView altında kusursuz entegrasyonu

Hiyerarşik Düzen

ImageSection + TitleSection + ButtonSection + TextSection sıralı dikey montajı tamamlandı.

Padding İzolasyonu

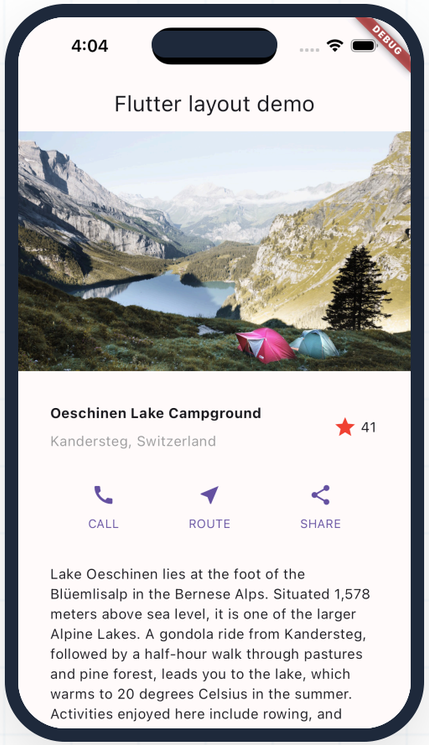
Her bileşen kendi iç boşluğunu yöneterek çevre bileşenlerle çakışmayı engeller.

Hot Reload Avantajı DevEx

Flutter'ın Hot Reload kabiliyeti sayesinde arayüzdeki stil değişiklikleri saniyeler içinde test edilir.

Etkileşim Hazırlığı

Bu statik yerleşim sonraki aşamada StatefulWidget ile beğeni sayacı gibi dinamik özelliklere kavuşabilir.



Tamamlanan Canlı Flutter Uygulaması

BÖLÜM 03 // GİRİŞ

03

İleri Seviye Listeler, Slivers & Paralaks Dinamikleri

ListView tembel yüklemesi, 2D GridView, Slivers kaydırma mimarisi ve Flow widget ile 60 FPS paralaks

ListView & GridView Mimarisi

Temel liste türleri, yatay listeler ve 2D ızgara dağılımları.

Performans & Bellek Yönetimi

ListView.builder tembel yükleme, prototypeItem ve itemExtent tasarrufu.

Slivers Mimarisi (CustomScrollView)

Viewport ve RenderSliver protokolü, SliverAppBar ve elastik kaydırma.

Flow ile Paralaks Efekti

Paint aşamasına müdahale, Matrix4 matris dönüşümleri ve sıfır jank.

Listelerle Çalışma: Temel ListView ve ListTile

Mobil uygulamaların kalbi olan dikey kaydırılabilir veri sunumu

ListView Nedir?

Column benzeri çalışan ancak içeriği ekrana sığmadığında otomatik dikey kaydırma sağlayan widget'tır.

Statik Liste Kullanımı

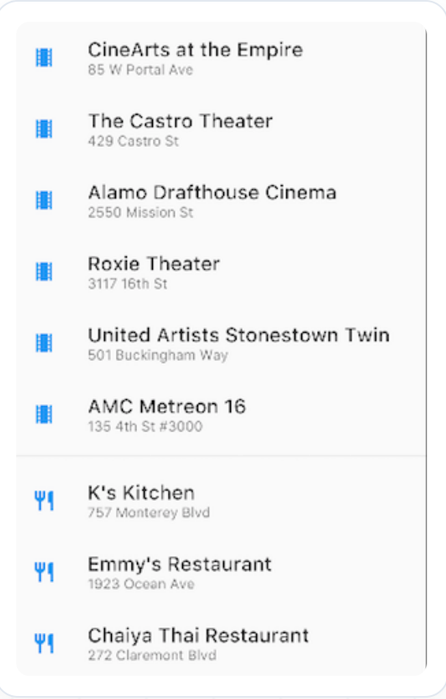
Az sayıda sabit öğe içeren ekranlar için doğrudan ListView(children: []) yapısı idealdir.

ListTile Entegrasyonu

Harita, albüm ve telefon gibi liste girdilerine standart Material görünümü kazandırır.

Divider Bileşeni

Farklı kategorilerdeki liste öğelerini görsel olarak birbirinden ayırmak için kullanılır.



ListTile ve Divider ile Ayrılmış Liste

İki Boyutlu Izgara Listeleri: GridView

GridView.count ve GridView.extent ile 2D esnek veri sunumu

GridView Mimarisi

Verileri iki boyutlu satır ve sütun matriksinde otomatik kaydırılabilir şekilde yerleştirir.

GridView.count

crossAxisCount parametresi ile ekranda kesin olarak kaç sütun bulunacağını sabitler.

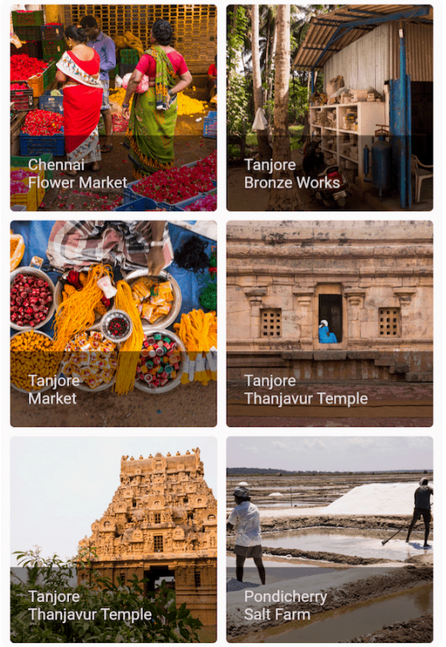
GridView.extent

Responsive

maxCrossAxisExtent ile her döşemenin azami piksel genişliğini belirler; sütun sayısı ekrana göre dinamik hesaplanır.

Görsel Galerileri

Ürün katalogları ve fotoğraf galerileri için GridView vazgeçilmez bir bileşendir.



GridView.count ile 2D Izgara Görünümü

Heterojen Listeler: Farklı Öğe Türlerini Sıralama

Polimorfizm ve soyut sınıf (abstract class) ile çok tipli listeler

Farklı Veri Tipleri

Aynı liste içinde hem kategori başlığı (HeadingItem) hem mesaj kartı (MessageItem) gösterme.

ListItem Arayüzü

Tüm öğelerin türediği ortak bir ListItem soyut sınıfı ve buildTitle/buildSubtitle metotları.

Dinamik Tip Eşleme (is Kontrolü)

itemBuilder içinde 'item is HeadingItem' kontrolüyle uygun widget'ı render etme.

Temiz Kod Ayrımı

Her öğe tipinin kendi çizim mantığını yönetmesi kod bakımını kolaylaştırır.

mixed_list.dart — DART

```dart
abstract class ListItem {
  Widget build(BuildContext context);
}

class HeadingItem implements ListItem {
  final String heading;
  HeadingItem(this.heading);

  @override
  Widget build(BuildContext context) =>
      Text(heading, style: Theme.of(context).textTheme.headlineSmall);
}

ListView.builder(
  itemCount: items.length,
  itemBuilder: (context, i) => items[i].build(context),
);
```

Aralıklı ve Esnek Liste Öğeleri (Spaced Items)

LayoutBuilder ve ConstrainedBox ile hem eşit boşluk hem kaydırma kabiliyeti

Problem: Spacer vs Scrollable

Kısıtlama

Kaydırılabilir widget'lar içinde Spacer doğrudan çalışmaz çünkü sonlu yükseklik kısıtı yoktur.

1. LayoutBuilder & constraints

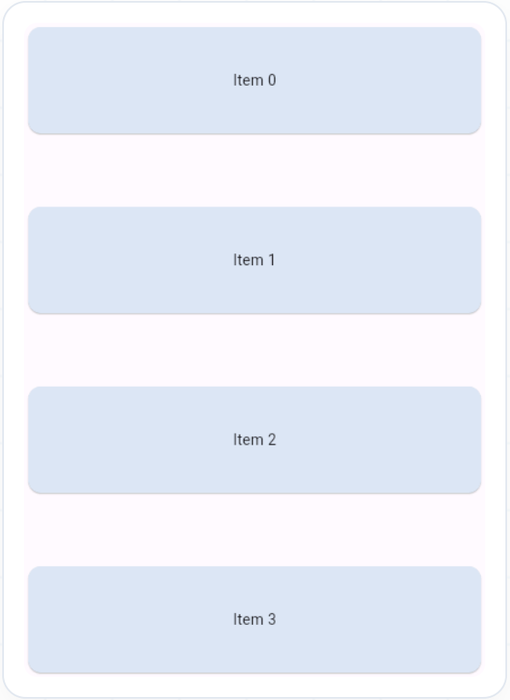
Üst widget'tan gelen BoxConstraints(maxHeight) bilgisini elde etmek için kullanılır.

2. ConstrainedBox(minHeight)

Alt içeriğin en az ekran boyu kadar olmasını garanti eder; böylece spaceBetween düzgün çalışır.

3. IntrinsicHeight

Öğelerin kendi doğal yüksekliklerine göre boyutlanıp kalan boşluğun Spacer ile esnemesini sağlar.



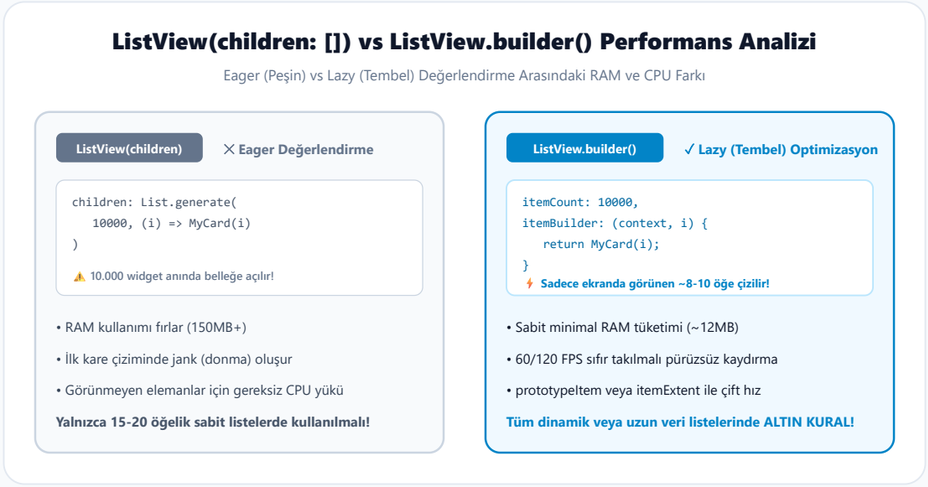
Spaced Items: Küçük Ekranda Kaydırma, Büyükte Eşit Dağılım

Yüksek Performanslı Uzun Listeler: Eager vs Lazy Builder

10.000+ elemanlı listelerde bellek tüketimi ve FPS karşılaştırması

ListView(children: []) Tuzağı

⚠ EAGER / BELLEK PATLAMASI



10.000 elemanın tamamını peşin (eager) belleğe açar. RAM fırlar (150MB+), ilk karede donma (jank) oluşur.

ListView.builder() Çözümü

✓ LAZY / 120 FPS ALTIN STANDART

long_list.dart — DART

```dart
final items = List<String>.generate(
  10000,
  (i) => 'Öğe $i',
);

ListView.builder(
  itemCount: items.length,
  itemBuilder: (context, index) {
    return ListTile(
      title: Text(items[index]),
    );
  },
);
```

Sadece ekranda görünen ~8-10 öğeyi tembel (lazy) render eder. Sabit minimal RAM (~12MB) ve 120 FPS pürüzsüz kaydırma.

Kaydırma Optimizasyonu ve Özel Bileşenler

prototypeItem/itemExtent tasarrufu, modal sayfalar ve 3D silindirik çark

Performans

prototypeItem & itemExtent

Öğelere sabit piksel (itemExtent: 72) veya prototip widget vererek CPU boyut ölçüm yükünü sıfırlar.

Modal

DraggableScrollableSheet

Ekranın altından yukarı sürüklenebilen ve parmak hareketine göre esneyen modal alt sayfalar.

3D Çark

ListWheelScrollView

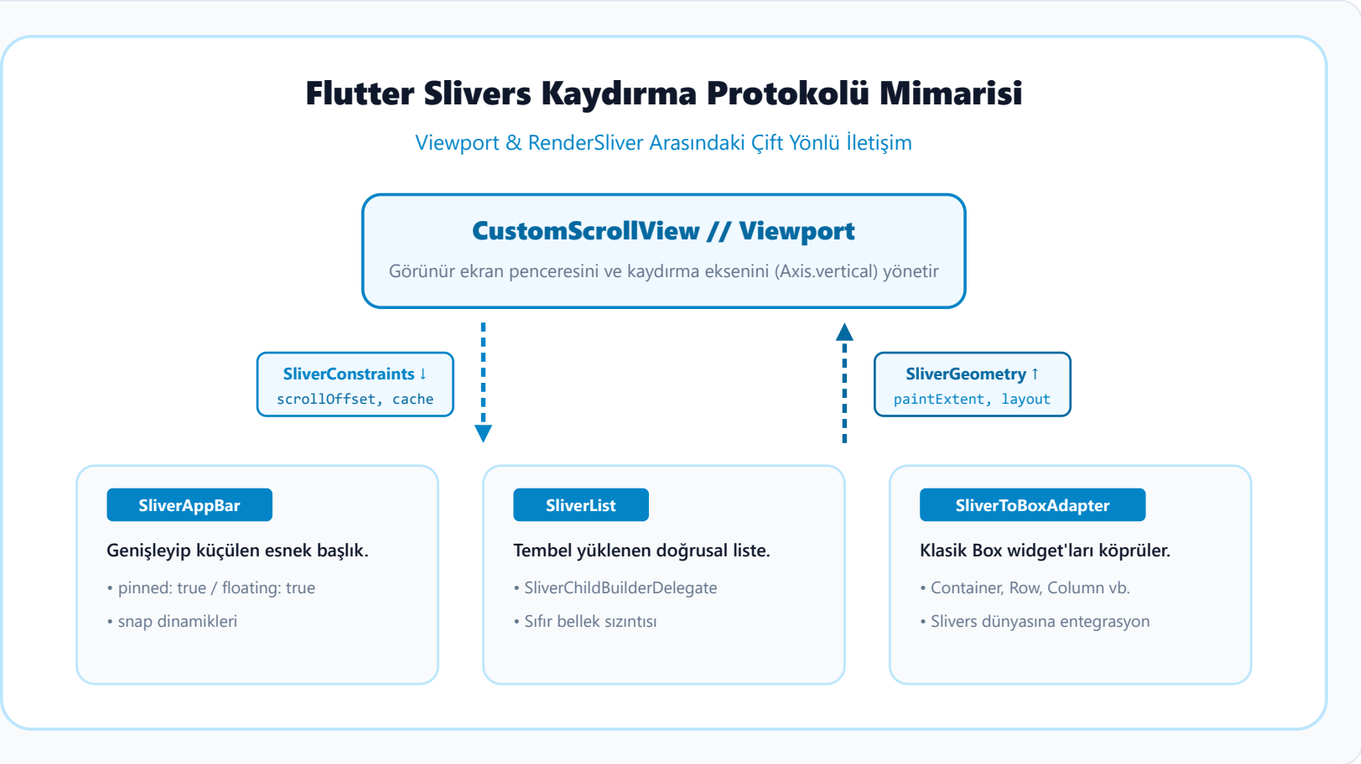
Öğeleri 3 boyutlu dönen silindirik bir çark yüzeyine yerleştirir; tarih/saat seçiciler için idealdir.

Scrollbar & ScrollNotification

Masaüstü ve mobilde tutulabilir kaydırma çubuğu ve milisaniyelik olay dinleyicisi (NotificationListener).

Gösterişli Kaydırma: CustomScrollView & Slivers Protokolü

Viewport ve RenderSliver arasındaki çift yönlü kısıtlama akışı



Sliver Nedir? Protokol

Kaydırılabilir alanın tek bir eksende (scroll axis) dilimlenmiş bağımsız render nesnesidir.

SliverConstraints ↓

Viewport aşağıya scrollOffset, remainingPaintExtent ve crossAxisExtent bilgilerini aktarır.

SliverGeometry ↑

RenderSliver yukarıya paintExtent (görünen alan) ve layoutExtent değerlerini bildirir.

SliverToBoxAdapter Köprü

Klasik kutu modellerini (Container, Row) Slivers dünyasına bağlayan evrensel köprüdür.

Performans Karşılaştırması: ShrinkWrap vs Slivers

İç içe kaydırılabilir yapılarda ölümcül bellek tuzağı ve mimari çözüm

ShrinkWrap Tuzağı

Dikkat

shrinkWrap: true, listenin lazy loading kabiliyetini öldürür ve tüm öğeleri tek seferde hesaplatır.

Slivers Mimarisi

Doğru Yol

CustomScrollView altındaki tüm sliver'lar tek bir kaydırma kontrolcüsünü paylaşır ve lazy render korunur.

CPU ve Pil Tüketimi

Büyük listelerde shrinkWrap kare atlamalarına (jank) ve cihazın ısınmasına yol açar.

Google Mühendisleri Tavsiyesi

Google ekibi iç içe kaydırmalarda istisnasız Slivers mimarisini tavsiye eder.

ÖZELLİK	SHRİNKWRAP: TRUE TUZAĞI	CUSTOMSCROLLVİEW + SLİVERS
Tembel Yükleme	İptal olur (Tüm elemanlar belleğe alınır) Kötü	Aktif kalır (Yalnızca görünenler çizilir) Mükemmel
Kaydırma Kontrolü	Ayrı ayrı bağımsız controller çakışması	Tek Viewport altında birleşik koordinasyon
Animasyon Desteği	Statik boyutlandırma, efekt desteği yok	Paralaks, esnek başlık, yapışkan öğeler
Büyük Veri Güvenliği	RAM taşması riski (OOM)	Sıfır sızıntı, kesintisiz 120 FPS

Dinamik Başlıklar: SliverAppBar Parametreleri

pinned, floating ve snap kombinasyonlarıyla zengin kullanıcı deneyimi

Sabit

pinned: true

Kullanıcı sayfayı ne kadar aşağı kaydırırsa kaydırsın, başlık çubuğu ekranın en üstünde sabit kalır.

Yüzen

floating: true

Kullanıcı yukarı doğru hafifçe kaydırdığı anda başlık listenin en başına dönmeyi beklemeden anında belirir.

Manyetik

snap: true

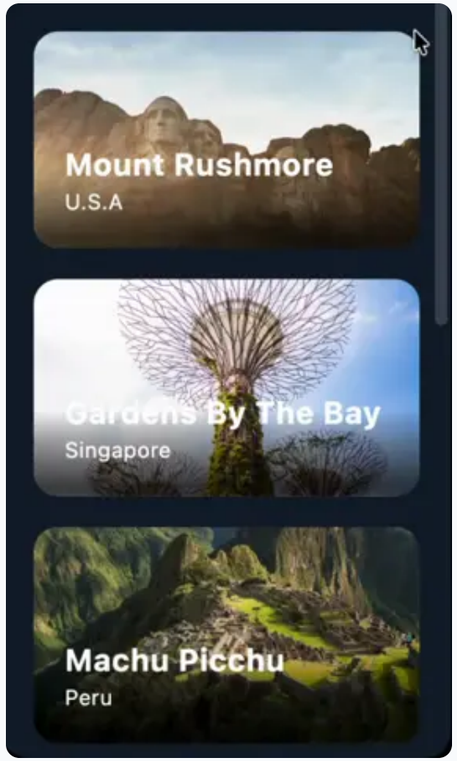
floating ile birlikte kullanılır; kullanıcı parmağını bıraktığı an başlık otomatik olarak ya tamamen açılır ya kapanır.

FlexibleSpaceBar

expandedHeight boyunca arka plan görselini, gradyanı ve başlık metnini pürüzsüzce küçültüp dönüştürür.

Kaydırmalı Paralaks Efekti: Parallax Scrolling & Flow Mimarisi

Arayüz derinliği için katmanların farklı hızlarda kaydırılması ve Paint aşaması



Görsel Derinlik Algısı

Ön plandaki kartlar ile arka plandaki görseller farklı hızlarda hareket ederek 3B gerçeklik hissi yaratır.

Paint Aşamasına Müdahale

Flow widget, çocuklarını boyutlandırırken layout'u bir kez yapar; kaydırma esnasında yalnızca paint() metodunu çağırır.

FlowDelegate ile Sıfır Rebuild

60 FPS

Doğrudan GPU matris dönüşümü yapıldığı için widget ağacı yeniden inşa edilmez; 60 FPS pürüzsüz akar.

CPU Layout Tasarrufu

Geleneksel setState() döngüsü yerine doğrudan boyama hattına bağlanarak takılmalar (jank) engellenir.

Paralaks Uygulaması: FlowDelegate, Matrix4 & Render Dönüşümü

Global piksel konumundan dikey kaydırma ofsetini türetme ve GPU dönüşümü

1. Scrollable ve Kart Konumu

ScrollableState üzerinden ekran sınırları, listItemContext üzerinden kartın anlık dikey yüksekliği yakalanır.

2. Yüzdelik Oran Hesabı Formül

scrollPercent = (listItemOffset.dy / (scrollableHeight - listItemHeight)) formülü işletilir.

3. Matrix4.translationValues

Görsel dikey eksende dX = 0, dY = (scrollPercent - 0.5) * maxOffset kadar anında kaydırılır.

4. RenderBox & ProxyBox Ultra Hız

Gereksiz BuildContext yeniden üretimlerini baypas ederek doğrudan RenderObject seviyesinde en yüksek hız sağlanır.

parallax_flow.dart — DART

```dart
class ParallaxFlowDelegate extends FlowDelegate {
  ParallaxFlowDelegate({
    required this.scrollable,
    required this.listItemContext,
    required this.backgroundImageKey,
  }) : super(repaint: scrollable.position);

  final ScrollableState scrollable;
  final BuildContext listItemContext;
  final GlobalKey backgroundImageKey;

  @override
  void paintChildren(FlowPaintingContext context) {
    // 1. Ekran ve kart konumlarını al
    final scrollBox = scrollable.context.findRenderObject() as RenderBox;
    final itemBox = listItemContext.findRenderObject() as RenderBox;
    final itemOffset = itemBox.localToGlobal(Offset.zero, ancestor: scrollBox);

    // 2. Yüzdelik oranı türet
    final scrollPercent = (itemOffset.dy / (scrollBox.size.height - itemBox.size.height)).clamp(0.0, 1.0);
```

BÖLÜM 04 // GİRİŞ

04

Uyarlanabilirlik, Büyük Ekranlar & Platform Adaptasyonu

Çoklu ekran boyut sınıfları, katlanabilir cihazlar, klavye/fare girdileri ve Material 3 vs Cupertino

Responsive vs Adaptive Felsefesi

Arayüzü sığdırmak (responsive) ile o cihaz bağlamında kullanışlı kılmak (adaptive).

Google'ın 3 Adımlı Modeli

Abstract (Soyutla) → Measure (Ölç) → Branch (Dallandır) mimari şablonu.

Büyük Ekran & Katlanabilirler

Window Size Classes, NavigationRail, letterboxing önleme ve Display API.

Çoklu Girdi & Platform Uyumu

Klavye kısayolları, fare imleçleri, VisualDensity ve .adaptive() kurucuları.

Google'ın 3 Adımlı Uyarlama Modeli: Abstract → Measure → Branch

Büyük ölçekli kurumsal uygulamalar için Google mühendislerinin önerdiği tasarım deseni

1

1. Soyutlama (Abstract)

Ekrandaki veri ve navigasyon modellerini görsel widget'lardan soyutlayın. Menü öğelerini tek bir bağımsız listede tanımlayın.

Adım 1: Soyutla

2

2. Ölçme (Measure)

Kullanılabilir ekran alanını MediaQuery.sizeOf(context) veya yerel kısıtlar için LayoutBuilder ile piksel piksel ölçün.

Adım 2: Ölç

3

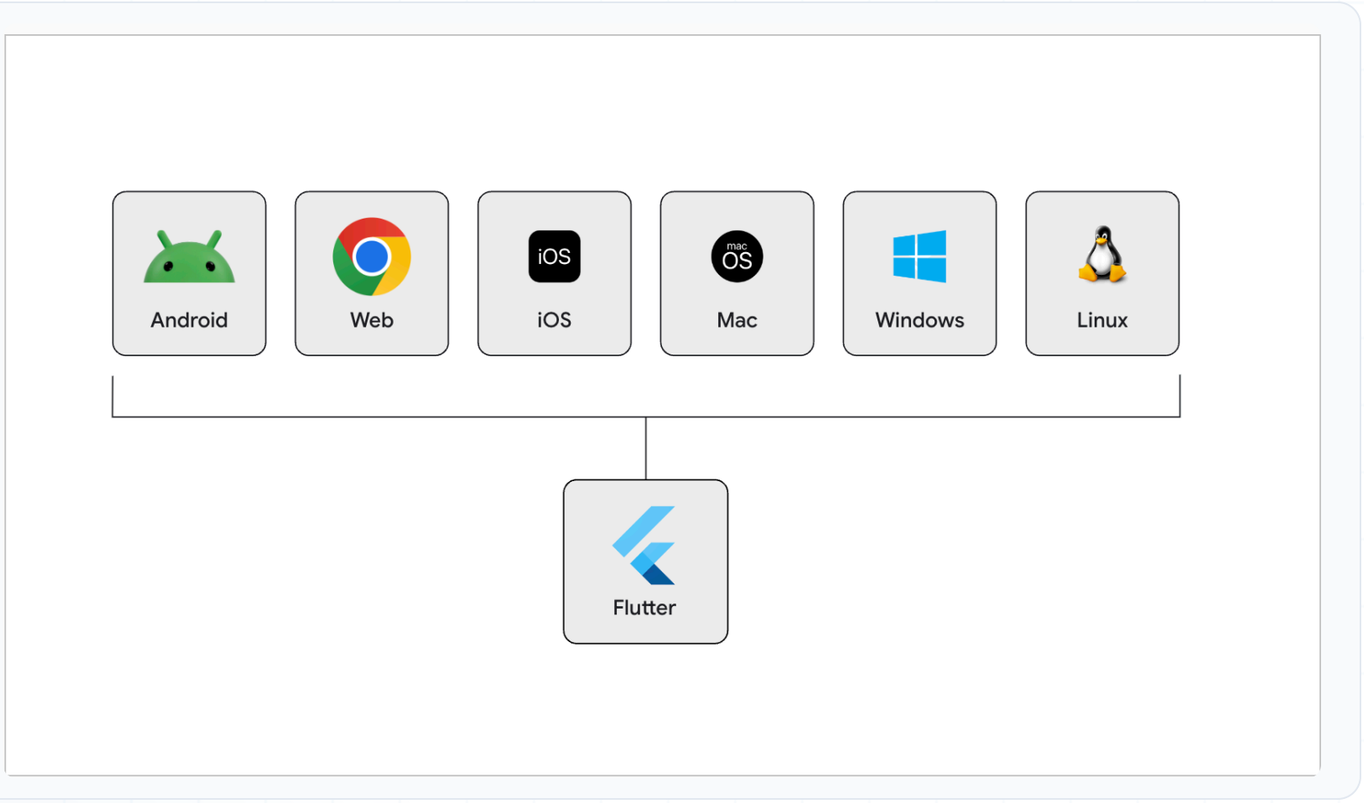
3. Dallandırma (Branch)

600dp kesme noktasına (breakpoint) göre mobilde NavigationBar, tablette NavigationRail şubesine dallandırın.

Adım 3: Dallandır

Çoklu Platform Ekosistemi: Tek Kod Tabanı, 6 Hedef

Android, iOS, Web, Windows, macOS ve Linux üzerinde tek bir arayüz mimarisi



Tek Kod Tabanı Vizyonu

Native

Flutter aynı iş mantığını ve UI kodunu hiçbir emülasyon olmadan yerel makine koduna derler.

Farklı Donanım Beklentileri

Mobil dokunmatik ekran beklerken, masaüstü fare imleci, klavye kısayolları ve çoklu pencere bekler.

Pencere Boyutuna Odaklanın

Cihaz adına değil pencere boyutuna göre karar verin; masaüstü penceresi daraltıldığında mobil düzenine geçebilmelidir.

Google Ekosistem Güvencesi

Resmi

6 platformun tamamı resmi Flutter motoru tarafından birinci sınıf vatandaş olarak desteklenir.

Ekran Ölçümü: MediaQuery.sizeOf vs LayoutBuilder

Neden MediaQuery.of yerine sizeOf kullanılmalı? Rebuild maliyet farkı

MediaQuery.sizeOf(context) Performans

Yalnızca boyut özelliğine abone olur; klavye açılması gibi diğer MediaQuery olaylarında gereksiz rebuild'i önler.

Global Pencere Boyutu

Uygulama penceresinin tamamının mantıksal piksel (logical pixel) cinsinden genişlik ve yüksekliğini verir.

LayoutBuilder: Yerel Kısıtlar

Üst widget'tan o noktaya aktarılan BoxConstraints (min/max aralıkları) bilgisini sağlar.

Doğru Aracı Seçmek

Tüm ekran düzeni için sizeOf; bağımsız bir bileşenin kendi alanı için LayoutBuilder kullanın.

measure_compare.dart — DART

```dart
// Global Pencere Kararı (Sayfa Düzeyi)
final windowSize = MediaQuery.sizeOf(context);
final isLargeScreen = windowSize.width >= 600;

// Yerel Kutu Kararı (Widget Düzeyi)
LayoutBuilder(
  builder: (context, constraints) {
    if (constraints.maxWidth > 400) {
      return TwoColumnLayout();
    }
    return SingleColumnLayout();
  },
);
```

SafeArea ve Donanım Çentikleri (Cutouts)

Kamera delikleri, yuvarlak köşeler ve durum çubuğunun arayüzü kesmesini engelleme

SafeArea Nedir?

Cihazın çentiklerini, kamera deliklerini ve durum çubuklarını otomatik kenar boşluğuna çevirir.

Kenar Bazlı Kontrol

İstenirse top, bottom, left veya right iç boşlukları bağımsız olarak devre dışı bırakılabilir.

Akıllı İç İçe İzolasyon

Zeka

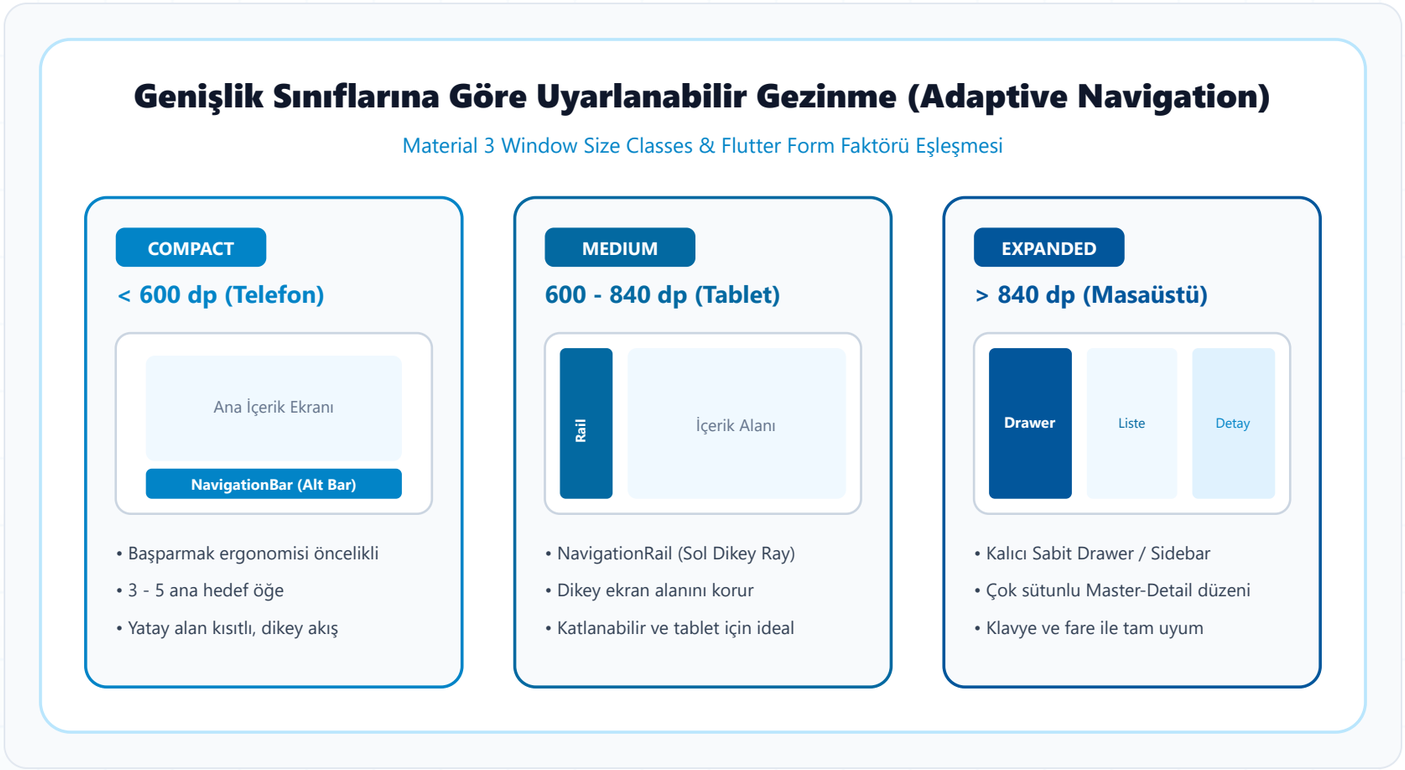
SafeArea altındaki widget'lara yeni bir MediaQuery sunar; böylece iç içe SafeArea'lar çifte boşluk üretmez.

Scaffold.body Sarmalama

Tüm Scaffold'u değil, yalnızca Scaffold'un body alanını sarmalamak AppBar'ın çubuk altına uzanmasını sağlar.

Genişlik Sınıflarına Göre Uyarlanabilir Gezinme (Adaptive Navigation)

Material 3 Window Size Classes: Compact, Medium ve Expanded eşleşmesi



Compact (< 600 dp) Telefon

Akıllı telefonlar için alt NavigationBar; başparmak ergonomisi ön plandadır.

Medium (600 - 840 dp) Tablet

Tablet ve katlanabilir cihazlar için dikey sol NavigationRail; dikey ekran alanını korur.

Expanded (> 840 dp) Masaüstü

Masaüstü ve geniş ekranlar için kalıcı sabit NavigationDrawer ve çok sütunlu Master-Detail düzeni.

Akıcı Geçiş

Pencere yeniden boyutlandırıldığında uygulama bu 3 mod arasında animasyonla pürüzsüzce geçer.

Katlanabilir Cihazlar (Foldables) & Letterboxing Tuzağı

Ekran yönünü kilitlemenin tehlikeleri ve Display API kullanımı

Letterboxing Tuzağı
Kritik Risk

Dikey moda kilitlenen uygulamalar katlanabilir cihaz açıldığında ekran ortasında siyah şeritlerle sıkışır.

Ekran Yönünü Asla Kilitlemeyin!

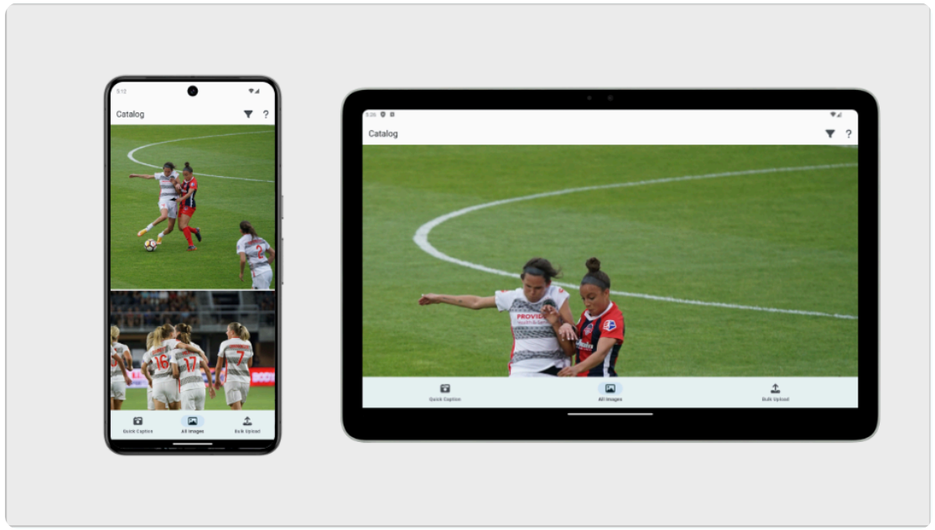
Android ve Flutter yönergeleri dikey kilit (portrait lock) yerine dinamik boyut yönetimini zorunlu kılar.

Display API Entegrasyonu

View.maybeOf(context)?.display üzerinden cihazın piksel oranına ve fiziksel yenileme hızına ulaşılır.

Menteşe (Hinge) Boşlukları

Katlanabilir donanımlarda ekran ortasındaki menteşenin arayüz öğelerini kesmemesi hesaplanmalıdır.



Geniş Ekran ve Katlanabilir Cihaz Düzeni

Uyarlanabilir Gezinme: NavigationBar vs NavigationRail

600dp kesme noktası kuralı: Mobilde alt bar, tablette sol dikey ray

Kullanıcı Ergonomisi

Mobilde başparmak alt kısma erişirken; tablet ve masaüstünde sol yan panel (rail) en doğal alandır.

600dp Standart Breakpoint

600dp Kuralı

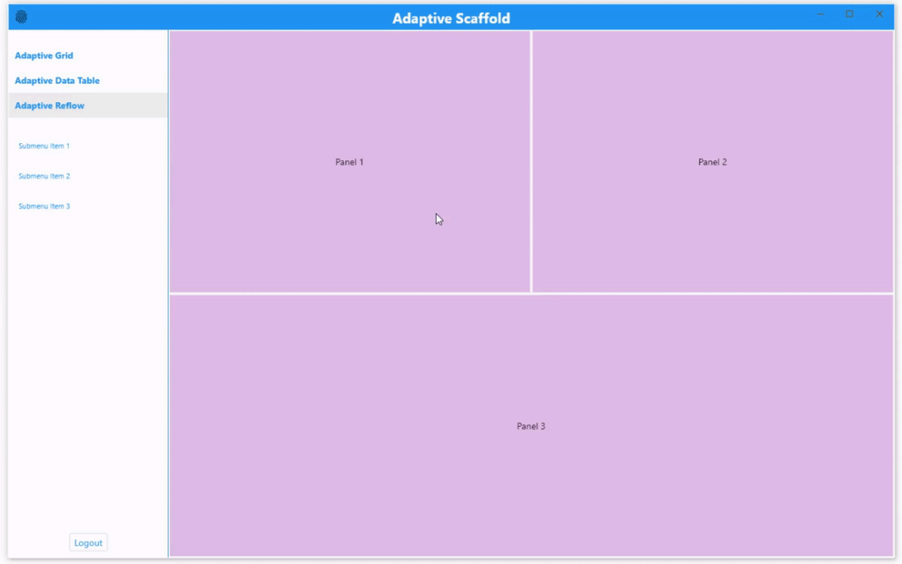
Genişlik < 600dp ise NavigationBar (alt bar); Genişlik >= 600dp ise NavigationRail (sol ray) çizilir.

Ortak Hedef Verisi

Menü öğeleri ve rota indeksleri tek bir liste üzerinden soyutlanarak kod tekrarı önlenir.

Row İçi Entegrasyon

Geniş ekranda NavigationRail solda sabit dururken sağ taraftaki içerik Expanded ile esnetilir.



Adaptive Scaffold: Ekran Boyutuna Göre Otomatik Dönüşüm

Masaüstü ve Web Girdi Modelleri: Fare, İmleç ve Klavye Kısayolları

MouseRegion, Listener, Tab gezinmesi (Focus) ve Shortcuts/Actions

İmleç

MouseRegion İmleçleri

Tıklanabilir alanlara gelindiğinde cursor: SystemMouseCursors.click ile el simgesi ve hover geri bildirimi sunulur.

PointerScrollEvent & Tekerlek

Listener ve onPointerSignal ile kaydırma tekerleğinin hızı ve yönü (scrollDelta.dy) milisaniyelik dinlenir.

Erişilebilirlik

FocusTraversalGroup & Tab Odak

Kullanıcı Tab tuşuna bastığında odağın ekran boyunca mantıksal bir sırayla gezmesini sağlar.

Kısayol

Shortcuts & Actions Mimarisi

Ctrl+N veya Shift+Enter gibi klavye tuş kombinasyonlarını Intent ve Action yapılarıyla iş mantığına bağlar.

Görsel Yoğunluk: VisualDensity

Dokunmatik ekranda geniş isabet alanı, masaüstünde kompakt veri sunumu

İsabet Alanı (Hit Area) Farkı

Parmakla dokunmada butonların geniş olması gerekirken, fare imlecinde kompakt öğeler tercih edilir.

VisualDensity Sınıfı

MaterialApp temasındaki görsel yoğunluğu yatay ve dikey eksende negatif veya pozitif değerlerle ayarlar.

Otomatik Pürüzsüz Geçiş

Otomatik

Yoğunluk değiştiğinde destekleyen Material bileşenleri yeni boyutlarına animasyonla pürüzsüzce geçer.

Theme.of(context).visualDensity

Özel widget'larınız da aktif yoğunluk değerini sorgulayarak kendi boyutlarını dinamik uyarlayabilir.

visual_density.dart — DART

```dart
// Dokunmatik modda geniş (0.0), fare modunda kompakt (-1.0)
double densityAmt = touchMode ? 0.0 : -1.0;

VisualDensity density = VisualDensity(
  horizontal: densityAmt,
  vertical: densityAmt,
);

MaterialApp(
  theme: ThemeData(visualDensity: density),
  home: const MainAppScaffold(),
);
```

Platforma Özgü Otomatik Davranışlar: Android vs iOS

Flutter motorunun arka planda işletim sistemine göre devreye aldığı yerel mekanikler

Doğal Platform Hissi

Flutter, geliştirici tek satır yazmasa dahi işletim sisteminin temel beklentilerini otomatik karşılar.

Sayfa Geçişleri (Transitions)

Android'de ZoomPageTransitions, iOS'ta ise yatay kayma ve arka planda paralaks geçişi çalışır.

Kaydırma Fiziği (Physics)

Android'de parıltılı (overscroll glow/stretch), iOS'ta ise yaylanmalı (bouncing) elastik kaydırma.

Tipografi & İkonlar

Android varsayılan olarak Roboto, iOS ise San Francisco yazı tipini ve platforma özel chevron ikonlarını kullanır.

PLATFORM ÖZELLİĞİ	ANDROİD (MATERİAL DESİGN)	İOS (CUPERTİNO HIG)
Sayfa Geçişi	Zoom / Alt-yukarı ölçekleme Android	Yatay Push & Kenardan kaydırıp geri gitme (Edge swipe) iOS
Kaydırma Fiziği	Clamping / Glow parıltısı	Bouncing / Elastik yaylanma ve momentum
En Üste Dönüş	Desteklenmez	Durum çubuğuna dokununca en üste kayma (Return to top)
Metin Seçimi	Tutamaçlı seçim ve Android toolbar	3D Touch kayan imleç ve iOS popover menü

Flutter UI İçin Altın Kurallar & Kapanış Özeti

Yüksek performanslı, ölçeklenebilir ve kusursuz çoklu platform geliştirme ilkeleri

1. Widget'ları Küçük Parçalara Ayırın (Modülerlik) Mimari

Büyük widget ağaçlarını bağımsız fonksiyon/sınıflara bölün ve 'const' kurucular kullanın; rebuild maliyetini sıfırlayın.

2. Doğru Kaydırma Aracını Seçin (Lazy Builder & Slivers) Performans

Uzun listelerde asla ListView(children) veya shrinkWrap kullanmayın; ListView.builder ve Slivers mimarisine sadık kalın.

3. Ekran Yönünü Asla Kilitlemeyin (Dinamik Uyarlanabilirlik) Adaptasyon

Katlanabilir cihazlar, tabletler ve masaüstü için MediaQuery.sizeOf ve Window Size Classes ile serbest kalın.

4. Liste Durumunu Koruyun (PageStorageKey) Kullanıcı Deneyimi

Ekran döndüğünde, pencere yeniden boyutlandığında veya sekmeler değiştiğinde kullanıcının kaydırma pozisyonunu saklayın.



✦ TEŞEKKÜRLER ✦

Bizi Dinlediğiniz İçin Teşekkür Ederiz

TAKIM ÜYELERİ

Furkan YAZGÜLÜ
Proje & Sunum Ekibi

Görkem Arda ÖNLER
Proje & Sunum Ekibi

Ege AYÇİÇEK
Proje & Sunum Ekibi

Takım Üyeleri: Furkan YAZGÜLÜ, Görkem Arda ÖNLER ve Ege AYÇİÇEK